Toolpath Verification, Glare Detection, and Zone Masking for Scan-N-Plan

README File

October 21, 2026

Abstract

This document explains how the `irdpl_toolpath_verifier` package works inside the Scan-N-Plan (SNP) framework. When 3D cameras scan shiny metallic parts, glare often blinds the sensor, leaving false holes in the surface mesh that cause the Noether path planner to stop early. At the same time, path planners treat the whole surface the same way, which can damage threads or alignment pins. This package sits between Noether and Tesseract: it checks if gaps are real holes or just camera glare using a small local vision model, bridges the missing path smoothly, and turns off the tool over protected areas while accounting for valve delay.

Contents

1 The Problems We Are Solving **2**

2 How It Fits into Scan-N-Plan **2**
- 2.1 Data Flow . . . 2
- 2.2 Integration Details . . . 2

3 Testing Local Vision Models **2**
- 3.1 Test Results . . . 2
- 3.2 Why We Picked Qwen 2.5-VL 3B . . . 2

4 Setup and Installation **4**
- 4.1 Requirements . . . 4
- 4.2 Setting Up the Local Vision Model . . . 4
- 4.3 Building the Package . . . 4
- 4.4 Starting the Service Node . . . 4

5 Adding It to the Behavior Tree **5**
- 5.1 Behavior Tree XML . . . 5
- 5.2 RViz Visualization . . . 5

6 Configuration Files **5**
- 6.1 Process Settings (`config/process_params.yaml`) . . . 5
- 6.2 Keep-Out Zones (`config/no_spray_zones.yaml`) . . . 5

1 The Problems We Are Solving

When running automated surface finishing or coating on metal parts, two common issues show up during 3D scanning and toolpath planning:

1. **Camera glare causes fake holes in the mesh:** Polished or turned steel parts reflect shop lights directly into the 3D camera. The active depth sensor gets saturated by the bright reflection and outputs empty readings (NaN). The reconstruction software turns these into long, fake holes across the part surface. When Noether tries to slice the surface to make toolpaths, it stops as soon as it hits these gaps, leaving fragmented, broken paths.
2. **Planners do not know which features to avoid:** Standard path planning algorithms treat every face equally. If a part has threaded ends, precision bearing shoulders, or locating pins, the robot will try to blast or coat right over them unless told otherwise.

This package acts as a bridge between path planning (Noether) and motion planning (Tesseract). It inspects the raw toolpath points against the scan, uses a local vision model to figure out whether a gap is real or just glare, fills in missing path points smoothly, and turns off the tool trigger over protected zones.

2 How It Fits into Scan-N-Plan

This package works as an add-on and does not require modifying Noether, Tesseract, or `industrial_reconstruc`
Everything connects through the standard Scan-N-Plan behavior tree in `snp_application`.

2.1 Data Flow

2.2 Integration Details

- **When it runs:** It triggers right after `industrial_reconstruction` builds the mesh and Noether generates the raw path slices.
- **How it is called:** It runs as a ROS 2 service at `/irdpl/verify_toolpath`, called directly by a Behavior Tree action node.
- **What it outputs:** It sends clean, continuous waypoints with spray trigger commands directly to Tesseract's planning service.

3 Testing Local Vision Models

Before choosing a model, we tested several small vision-language models locally on the edge computer. We wanted to see if they could tell the difference between actual drilled holes and bright glare streaks on curved metal parts.

3.1 Test Results

3.2 Why We Picked Qwen 2.5-VL 3B

Interestingly, the larger models (`qwen2.5vl:7b` and `granite3.2-vision`) were overly cautious: whenever they saw a bright glare streak on a solid steel rod, they guessed it was a hole.

`qwen2.5vl:3b` got every test right, reliably returned clean JSON that our Python node could parse, and answered in under a second on local hardware. We chose it as the default model.

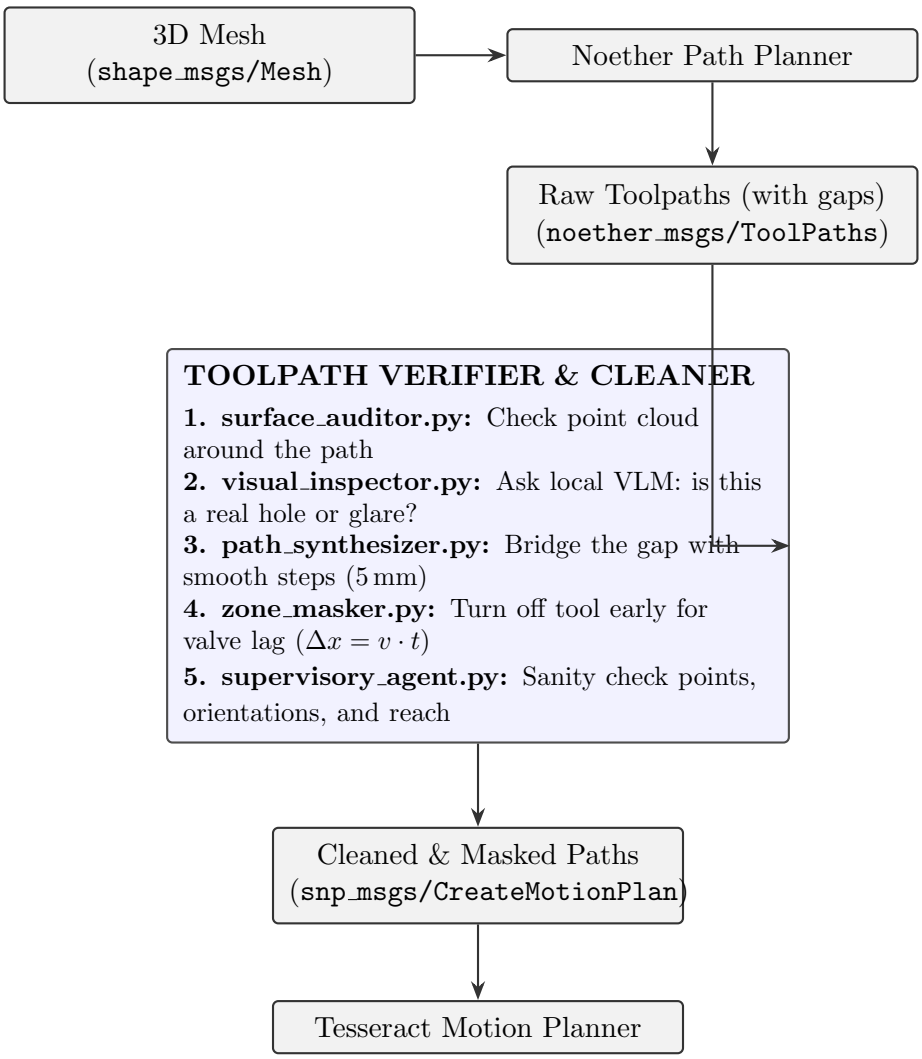


Figure 1: How data flows through the verifier before motion planning.

Model	Accuracy	Handled Glare?	Valid JSON?	Decision
`qwen2.5vl:3b`	**100%**	Correctly identified glare	Yes (100%)	**Selected**
`qwen2.5vl:7b`	67%	Mistook glare for holes	Yes (100%)	Rejected
`granite3.2-vision`	67%	Mistook glare for holes	Yes (95%)	Rejected
`moondream:latest`	Failed	Inconsistent answers	No (0%)	Rejected

Table 1: Local model test results on shiny metal test pieces.

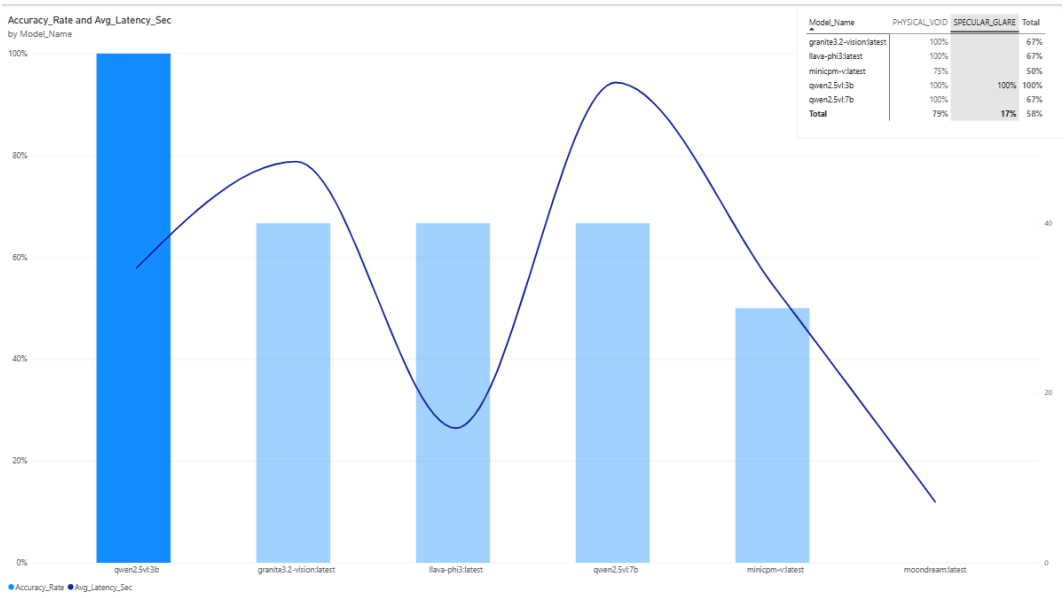


Figure 2: Benchmark evaluation and visual performance analysis across tested local vision models.

4 Setup and Installation

4.1 Requirements

This package runs on Ubuntu 22.04 LTS with ROS 2 Humble. Install the required Python packages:

```
sudo apt update && sudo apt install python3-pip -y
pip install open3d scipy numpy scikit-learn
```

4.2 Setting Up the Local Vision Model

Install Ollama and pull the 3B model so the robot can run visual checks completely offline:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &
ollama pull qwen2.5vl:3b
```

4.3 Building the Package

Clone this repo into your ROS 2 workspace and build it:

```
cd ~/ros2_ws
colcon build --symlink-install --packages-select irdpl_toolpath_verifier
source install/setup.bash
```

4.4 Starting the Service Node

Launch the node with its default config files:

```
ros2 launch irdpl_toolpath_verifier toolpath_verifier.launch.py
```

5 Adding It to the Behavior Tree

5.1 Behavior Tree XML

Open `snp_application/config/behavior_tree.xml` and add the verification action between the path planning and motion planning steps:

```
<Action ID="VerifyToolpath" service_name="/irdpl/verify_toolpath"/>
```

5.2 RViz Visualization

The node publishes visual markers to `/irdpl/verified_toolpath_markers` so you can see what it did in RViz:

- **Green:** Normal working path at the correct standoff distance.
- **Yellow:** Path that was bridged across a glare gap.
- **Red:** Protected zone (tool trigger is turned off here).

6 Configuration Files

6.1 Process Settings (`config/process_params.yaml`)

- `nominal_standoff`: Target distance from the tool tip to the part surface (z-offset).
- `interpolation_step`: Distance between generated waypoints when bridging a gap (default is 5.0 mm).
- `valve_lag_time`: How long the pneumatic valve takes to physically close (t_{lag}). The code shuts off the trigger early based on tool speed:

$$\Delta x = v_{\text{tool}} \cdot t_{\text{lag}} \tag{1}$$

6.2 Keep-Out Zones (`config/no_spray_zones.yaml`)

- `geometric_bounds`: Bounding boxes around areas that must not be coated or blasted (such as threads and alignment pins).
- `frame_id`: The coordinate frame the bounding box is tied to.